

L7. Quantum Algorithms

Xin Wan

Zhejiang University

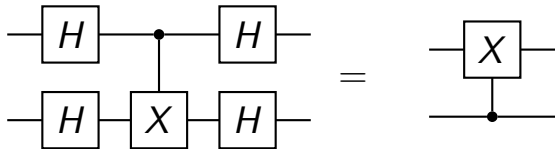
Outline

- The Deutsch Algorithm
- The Grover Search Algorithm
- Quantum Supremacy

7.1 Deutsch's Trick

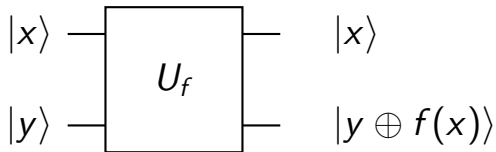
Take-Home Messages

- We will now see a concrete example that quantum computing can have an advantage, although tiny, over classical computing.
- Equivalent circuits can be useful in understanding quantum algorithms. For example, $H^2 = 1$, $HXH = Z$, and



The Deutsch Problem (1985)

- There exist only four different 1-qubit functions:

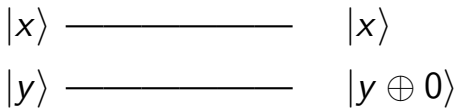


- The four functions are 0, x , $\bar{x}$, and 1. Or, explicitly,
 - Notice that, with the truth table or the Venn diagram, one can prove

$$A \oplus B = \bar{A} \oplus \bar{B}.$$

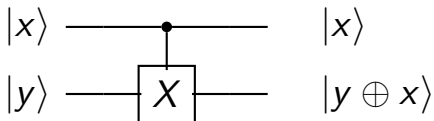
(i) $U_f = I_4$:

$$f_1(0) = f_1(1) = 0.$$



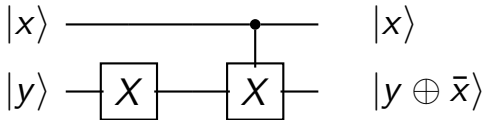
(ii) $U_f = \text{CNOT}$:

$$f_2(0) = 0, \quad f_2(1) = 1.$$



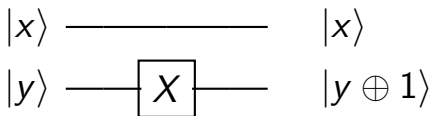
(iii) $U_f = \text{CNOT}(I_2 \otimes X)$:

$$f_3(0) = 1, \quad f_3(1) = 0.$$



(iv) $U_f = (I_2 \otimes X)$:

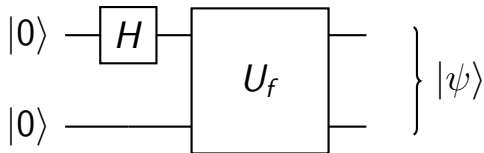
$$f_4(0) = f_4(1) = 1.$$



- Suppose that we are given a blackbox that executes U_f for one of the four functions. What can we learn about f , if we let the box act once?
- In the classical case:
 - We either learn $f(0)$, or $f(1)$. For example, if $f(0) = 0$, we know $f = f_1$ or f_2 . Otherwise, $f = f_3$ or f_4 .
 - If we want to fix f , we have to run the box twice, such that we know $f(0)$ and $f(1)$.
 - If we want to know whether $f(0) = f(1)$, which is a 1-bit information, we still need to run the box twice.
- Remarkably, with a quantum computer we do not have to run U_f twice to determine whether or not f is a constant.

An Intuitive Trial

- The standard trick, as we discussed on quantum parallelism, is to put the input qubit into the equal superposition of $|0\rangle$ and $|1\rangle$ before the application of U_f .



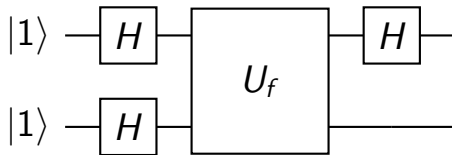
- We obtain

$$|\psi\rangle = \frac{1}{\sqrt{2}}(|0\rangle |f(0)\rangle + |1\rangle |f(1)\rangle).$$

- By applying Hadamards to both qubits we obtain
 - $f_1(0) = f_1(1) = 0$, $H \otimes H |\psi\rangle = \frac{1}{\sqrt{2}} |0\rangle (|0\rangle + |1\rangle)$;
 - $f_2(0) = 0$, $f_2(1) = 1$, $H \otimes H |\psi\rangle = \frac{1}{\sqrt{2}} (|00\rangle - |11\rangle)$;
 - $f_3(0) = 1$, $f_3(1) = 0$, $H \otimes H |\psi\rangle = \frac{1}{\sqrt{2}} (|00\rangle + |11\rangle)$;
 - $f_4(0) = f_4(1) = 1$, $H \otimes H |\psi\rangle = \frac{1}{\sqrt{2}} |0\rangle (|0\rangle - |1\rangle)$.
- Now measure both qubits. Half the time we obtain 00 and know nothing. Half the time, we either get 01, so $f(0) = f(1)$, or 11, so $f(0) \neq f(1)$. We have a 50% chance of success; one cannot do any better by replacing $H \otimes H$ by other operators.

The Deutsch Algorithm

- A 100%-effective method is the following.



- The result is

$$|\psi\rangle = \begin{cases} |1\rangle \frac{1}{\sqrt{2}} [|f(0)\rangle - |1 \oplus f(0)\rangle], & f(0) = f(1) \\ |0\rangle \frac{1}{\sqrt{2}} [|f(0)\rangle - |1 \oplus f(0)\rangle], & f(0) \neq f(1) \end{cases}$$

which can be distinguished by measuring the input qubit.

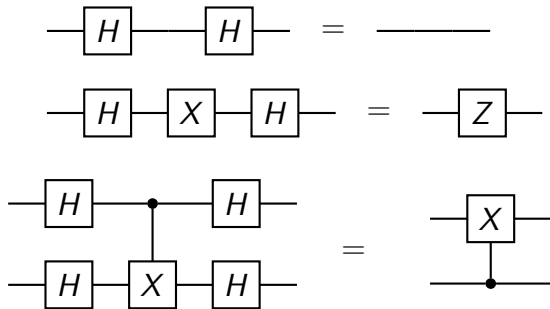
- The final state of the output qubit is

$$\begin{cases} \frac{1}{\sqrt{2}} [|0\rangle - |1\rangle], & f(0) = 0 \\ \frac{1}{\sqrt{2}} [|1\rangle - |0\rangle], & f(0) = 1 \end{cases}$$

Therefore, we do not have any information on $f(0)$, because we always measure $|0\rangle$ or $|1\rangle$ with equal probabilities. The sign difference is not detectable.

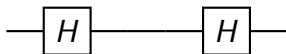
- In some sense, this is fair. We have applied U_f only once and eliminated two of the four possibilities.
- Remarkably, this particular discrimination cannot be done by any classical computer with only one application of U_f .

- Recall there are equivalent circuits in quantum circuits.

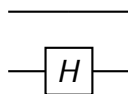


- See if you can use them to obtain the equivalent circuits on next page. From the equivalent circuits, can you understand why Deutsch's algorithm works?

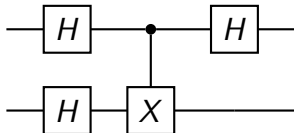
(i) $f(x) = 0$



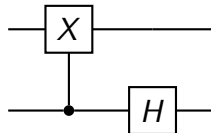
=



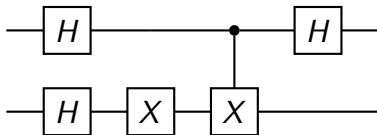
(ii) $f(x) = x$



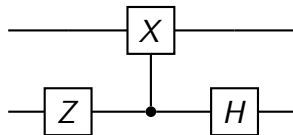
=



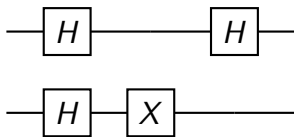
(iii) $f(x) = \bar{x}$



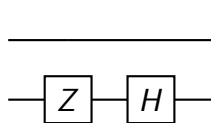
=



(iv) $f(x) = 1$



=



7.2 Grover's Trick

Take-Home Messages

- Grover's algorithm searches for an entry in an unstructured database and, thus, can be practically useful. However, the speed-up only goes from $O(N)$ to $O(\sqrt{N})$.
- Geometrically, the algorithm rotates a state toward the target through a series of reflections.
- If we run the algorithm once, we only have a very high probability to obtain the desired result. To ensure the success, we need to repeat the computation a few times.

The Grover Search Algorithm

- The Grover search algorithm performs a search for an entry in an unstructured database, e.g., the title of a book in a library ordered by first author's name.
- The normal setup of the problem assumes that we have been given a subroutine that returns

$$f(x) = 0, \quad x \neq a; \quad f(x) = 1, \quad x = a,$$

for any n -bit integer x , where a is the special value being sought.

- In practice, it is told that we are given a black box, which is often called an oracle, containing a quantum circuit (which we do not know in general) that implement the function.
- The oracle incorporates the usual quantum-computational trick that applies a unitary transformation U_f on an n -qubit input and a 1-qubit output as

$$U_f(|x\rangle_n |y\rangle_1) = |x\rangle_n |y \oplus f(x)\rangle_1,$$

such that the output is not flipped or is flipped from 0 to 1 (or vice versa), depending on whether $x = a$ or not.

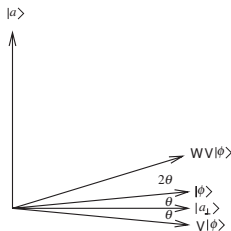
- We are supposed to start from some quantum state. By repeatedly calling the oracle, we manage to evolve the state toward the unknown $|a\rangle$ —so we find it.
- If the number of entries in the database is N , which can be encoded by n qubits (i.e., $N \lesssim 2^n$). A classical algorithm (checking one by one) on average makes $N/2$ attempts to find the book title.
- The Grover algorithm, as we will show, can solve the problem in $\sim \sqrt{N}$ operations. This is an appreciable, though not exponential, speed-up.

Grover's Algorithm: Geometrical Picture

- How does Grover's search works in principle? We use vectors in a two-dimensional plane to illustrate the algorithm. Suppose $|a\rangle$ is the state we want to search. Define $|a_{\perp}\rangle$ to be the state orthogonal to $|a\rangle$.
- Let us introduce an auxiliary state

$$|\phi\rangle = \cos \theta |a_{\perp}\rangle + \sin \theta |a\rangle,$$

where θ is the angle between $|\phi\rangle$ and $|a_{\perp}\rangle$.



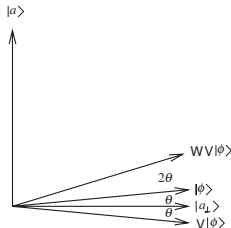
- We introduce a reflection V with respect to the direction of $|a_{\perp}\rangle$ as

$$V = 2|a_{\perp}\rangle\langle a_{\perp}| - I = I - 2|a\rangle\langle a|.$$

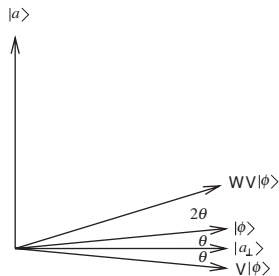
The operation flips the sign of the $|a\rangle$ component, while leaving unchanged the components perpendicular to $|a\rangle$.

- We can also introduce a reflection W with respect to the direction of $|\phi\rangle$ as

$$W = 2|\phi\rangle\langle\phi| - I.$$



- The product of the two reflections is a rotation: WV rotates a vector through the angle 2θ in the direction from $|a_{\perp}\rangle$ to $|\phi\rangle$.



- Each subsequent application of WV increases the angle between the final state and $|a_{\perp}\rangle$ by another 2θ . A proper choice of the number of WV applications can approach $|a\rangle$ with an error in angle no more than θ .

How to Relate V to the Oracle?

- How does the oracle fit into the algorithm that repeatedly rotates a quantum state? The answer is that the reflection of V can be straightforwardly implemented with a call of the black box U_f .
- Notice that

$$V|x\rangle = \begin{cases} |x\rangle, & x \neq a, \\ -|a\rangle, & x = a. \end{cases}$$

This is nothing but $(-1)^{f(x)}|x\rangle$, so we need a trick to make the effect of U_f merely a sign change.

- This is done by initializing the output qubit into the state

$$H|1\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$$

before the application of U_f .

- Convince yourself that

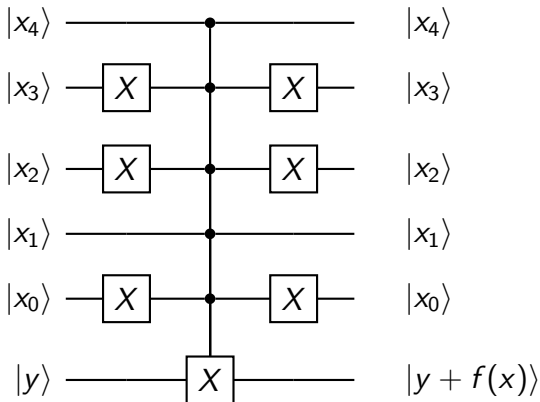
$$U_f(|x\rangle \otimes H|1\rangle) = (-1)^{f(x)} |x\rangle \otimes H|1\rangle.$$

- Interestingly, the application of U_f on the output qubit can be regarded as the identity, so it is never entangled with the input qubits and remains a constant state throughout the algorithm.

- Therefore, for an arbitrary n -qubit input state $|\Psi\rangle = \sum_x \alpha_x |x\rangle$, we have explained how to implement

$$U_f(|\Psi\rangle \otimes H|1\rangle) = [|\Psi\rangle - 2|a\rangle \langle a|\Psi\rangle] \otimes H|1\rangle.$$

- For completeness, we cite the example in Mermin's book for a possible realization of a black box that implements U_f with $a = 10010$.



How to Choose $|\phi\rangle$ and Construct W ?

- Now, the construction of W is closely tied to the choice of the auxiliary state $|\phi\rangle$. What would you choose?
- Of course, we expect the algorithm should work for any n -qubit $|a\rangle$. So, $|\phi\rangle$ should not be biased toward any n -qubit basis vector. We can choose, e.g.,

$$|\phi\rangle = H^{\otimes n} |0\rangle_n = \frac{1}{2^{n/2}} \sum_{x=0}^{2^n-1} |x\rangle_n.$$

- $|\phi\rangle$ is easy to prepare.
- $|\phi\rangle$ is almost orthogonal to any $|x\rangle_n$.

- We may as well consider the construction of $-W$

$$-W = 1 - 2|\phi\rangle\langle\phi| = H^{\otimes n}(1 - 2|00\dots 0\rangle\langle 00\dots 0|)H^{\otimes n}.$$

Notice that $H^{\otimes n}$ is its own inverse.

- Now we need an operation $W' = 1 - 2|00\dots 0\rangle\langle 00\dots 0|$, which obeys

$$W'|x\rangle_n = \begin{cases} |x\rangle_n, & x \neq 00\dots 0, \\ -|x\rangle_n, & x = 00\dots 0. \end{cases}$$

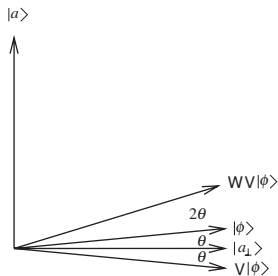
- One can verify that

$$W' = X^{\otimes n}(C^{n-1}Z)X^{\otimes n},$$

where $C^{n-1}Z$ is the $(n-1)$ -fold controlled- Z gate.

Performance by Geometrical Analysis

- As we discussed in the geometrical picture, each application of WV reduces the angle between the resulting state and $|a\rangle$ by 2θ .
- With our choice of $|\phi\rangle$,



$$\cos\left(\frac{\pi}{2} - \theta\right) = \langle a|\phi\rangle = 2^{-n/2} \approx \frac{1}{\sqrt{N}}.$$

- For large $\sqrt{N}$, it is fairly accurate that $\theta \approx 2^{-n/2}$.

- To rotate $|\phi\rangle$ to $|a\rangle$, we can apply an integral number of WV s. A proper choice of the number is as close as possible to

$$\frac{\pi}{2}/(2\theta) = \frac{\pi}{4}2^{n/2} \sim \sqrt{N},$$

and the error in angle is no more than $\theta \sim 1/\sqrt{N}$.

- If we now measure the input qubits, we have a very high probability to obtain the desired $|a\rangle$. We can repeat the whole procedure a few times to make sure we successfully find $|a\rangle$ in a practical sense.

7.3 Quantum Supremacy

Take-Home Messages

- Random circuit sampling is among a set of problems that the present-day quantum computers can demonstrate supremacy over classical computers, on which the algorithm is intractable.

Quantum Supremacy

- **Quantum supremacy**, coined by John Preskill, refers to a computational task that can be efficiently performed on a quantum computer beyond the abilities of state-of-the-art classical supercomputer can efficiently implement.
- This, however, does not necessarily mean that the computational task is useful. Nevertheless, it is a milestone in quantum computing, demonstrating a clear-cut ability to run efficiently on a quantum processor compared with a classical computer where the algorithm is intractable.

- Supremacy experiments on a quantum computer have now refuted the extended Church-Turing thesis which asserts that any algorithmic process can be simulated efficiently using a probabilistic Turing machine. The engineering achievement leads us, instead, to the quantum extended Church-Turing thesis.
- We will briefly discuss the problem implemented by F. Arute et al. in Quantum supremacy using a programmable superconducting processor [Nature 574, 505 (2019)].

Random Circuit Sampling

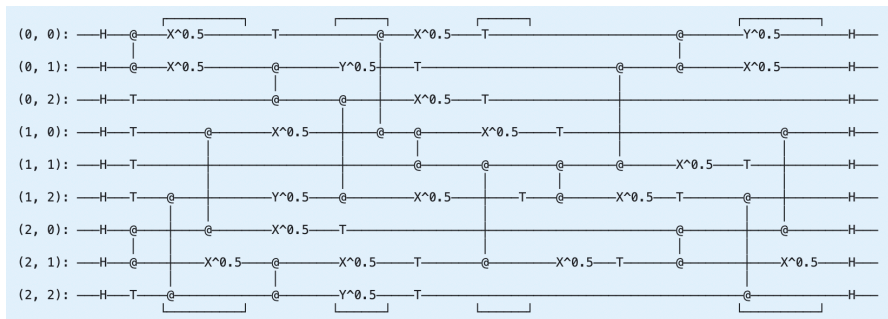
- The problem samples from the output distribution of a random quantum circuit.¹
- Classical methods for simulating quantum circuits generally scale exponentially in the number of qubits. Due to memory constraint, the current limit is about 50 qubits.

¹see, e.g., J. D. Hidary, Quantum Computing: An Applied Approach, 2nd Ed. (Springer, 2021), Chap. 10.2 for detail.

```

import cirq
nrows, ncols, depth = 3, 3, 10
supremacy_circuit = cirq.experiments.
    generate_boixo_2018_supremacy_circuits_v2_grid
    (nrows, ncols, depth, seed=123)
print(supremacy_circuit)

```



Summary

- With the development of more and more powerful quantum processors, our interest has shifted from toy algorithms that demonstrate the advantages of a quantum computer to algorithms showing quantum supremacy.
- The ongoing research trend is to develop quantum computing programs, in particular in quantum chemistry, optimization, linear equations, and machine learning, that can be run on noisy processors.

References

- N. D. Mermin, Quantum Computer Science: An Introduction (Cambridge, 2007), Chap. 4.
- J. D. Hidary, Quantum Computing: An Applied Approach, 2nd Ed. (Springer, 2021), Chap. 8.6.

Homework

P7-1. The quantum Fourier transform can be regarded as a mapping

$$\sum_{q=0}^{N-1} x_q |q\rangle \longrightarrow \sum_{k=0}^{N-1} y_k |k\rangle, \quad y_k \equiv \frac{1}{\sqrt{N}} \sum_{q=0}^{N-1} x_q e^{2\pi i k q / N}.$$

Explicitly write down the mapping for the case of $n = 1$ qubit. What is the quantum gate that realizes the qubit-qubit mapping?

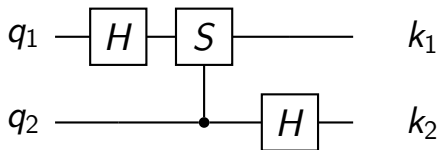
For $n = 1$ and $N = 2$, we have

$$x_0 |0\rangle + x_1 |1\rangle \longrightarrow \frac{1}{\sqrt{2}}(x_0 + x_1) |0\rangle + \frac{1}{\sqrt{2}}(x_0 - x_1) |1\rangle .$$

They are related by a **Hadamard gate**, as

$$H(x_0 |0\rangle + x_1 |1\rangle) = x_0 \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) + x_1 \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle).$$

P7-2. Consider the following quantum circuit for two qubits.



For each basis state $|q_1 q_2\rangle = |00\rangle, |01\rangle, |10\rangle, |11\rangle$, write down the output $|k_1 k_2\rangle$. Are they the same as you expect from the quantum Fourier transform? Generalize the circuit to perform the quantum Fourier transform for $n = 3$ qubits. Make proper changes, if necessary, according to your two-qubit result.

The whole circuit implements

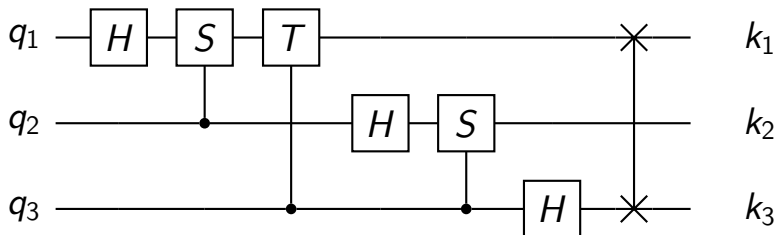
$$\begin{aligned} U &= \begin{bmatrix} H & 0 \\ 0 & H \end{bmatrix} \begin{bmatrix} I & 0 \\ 0 & S \end{bmatrix} \frac{1}{\sqrt{2}} \begin{bmatrix} I & I \\ I & -I \end{bmatrix} = \frac{1}{\sqrt{2}} \begin{bmatrix} H & H \\ HS & -HS \end{bmatrix} \\ &= \frac{1}{2} \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & -1 & 1 & -1 \\ 1 & i & -1 & -i \\ 1 & -i & -1 & i \end{bmatrix}. \end{aligned}$$

The input-output states are

- $|00\rangle \longrightarrow \frac{1}{2}(|00\rangle + |01\rangle + |10\rangle + |11\rangle)$
- $|01\rangle \longrightarrow \frac{1}{2}(|00\rangle - |01\rangle + i|10\rangle - i|11\rangle)$
- $|10\rangle \longrightarrow \frac{1}{2}(|00\rangle + |01\rangle - |10\rangle - |11\rangle)$
- $|11\rangle \longrightarrow \frac{1}{2}(|00\rangle - |01\rangle - i|10\rangle + i|11\rangle)$

They are not. Would be the same if $|01\rangle$ and $|10\rangle$ were exchanged. In other words, we need an extra **SWAP** gate to implement the quantum Fourier transform.

We need three gates (H , S , T) for the additional qubit. We also need to swap qubit 1 and 3 to reverse the order of k_i .



P7-3. Quantitatively compare the classical and quantum complexity of the Fourier transform. That is, how does the number of gates needed to implement the algorithm depends on the problem size (in this case the number of bits/qubits n)?

Classical complexity:

We need to compute N coefficients y_k , each having a sum of N terms (a multiplication of number and an exponential).

Therefore, we need N^2 multiplications, i.e., $\mathcal{O}(2^{2n})$. There exists a Fast Fourier Transform algorithm, which uses $\mathcal{O}(n2^n)$ gates. But this is still exponential in n .

Quantum complexity:

Acting on the lowest qubit, there is only one Hadamard gate. On the second lowest, there are two gates, one Hadamard and one controlled rotation. Next, one Hadamard and two controlled rotations. We thus expect for n qubits, we need a total of n Hadamard gates and

$$0 + 1 + 2 + \cdots + (n - 1) = n(n - 1)/2$$

controlled rotations. We also need up to $n/2$ swap gates, each of which can be implemented by 3 CNOT gates. Therefore, the circuit provides an $\mathcal{O}(n^2)$ algorithm to execute quantum Fourier transform.